

Angular Learning Series

Advanced Topics — Consolidated Reference

Custom Directives | State Management Patterns | RxJS Operators in Depth
Feature-based Folder Structure | Testing Angular Components
CDK & Animations | Performance Optimisation

June 2026

Custom Directives

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The decorator and class structure are straightforward, but understanding when to build a directive versus a component, and how to correctly interact with the host element without breaking Angular's rendering model, requires deliberate practice.

Angular Relevance: High — Custom directives are the correct tool for reusable DOM behaviour that doesn't need its own template — click-outside detection, auto-focus, permission-based visibility, tooltip anchoring, and input formatting all belong in directives, not components.

Section 1 — Explanation

The Sticker Sheet Analogy

Imagine you're decorating notebooks. A component is like printing a completely new page and inserting it — it brings its own content, its own layout, its own structure.

A directive is like a sticker you apply to an existing page — it adds behaviour, changes appearance, or reacts to interaction, but the page itself already exists. The sticker doesn't replace the page; it enhances it.

- A tooltip sticker — hover over the page, a bubble appears
- A highlight sticker — the text underneath turns yellow
- A lock sticker — you can't write on this section unless you have a key

You can put multiple stickers on the same page. Each sticker does one focused job. The page (host element) doesn't know or care how many stickers it has — it just behaves according to all of them.

In Angular terms:

- The host element is the notebook page — it already exists in the template
- Your directive class is the sticker — it attaches to that element via a CSS selector
- `@HostListener` is the sticker reacting to what happens on the page (hover, click, keypress)
- `@HostBinding` is the sticker changing something about the page's appearance or attributes

Technical Explanation

A directive is an Angular class decorated with `@Directive` that attaches to a DOM element matched by a CSS selector. Unlike components, directives have no template of their own — they operate on the host element they are placed on.

Three types of directives:

Type	Purpose	Examples
Attribute directive	Changes appearance or behaviour of an existing element	<code>ngClass</code> , <code>ngStyle</code> , your custom <code>appHighlight</code>

Structural directive	Changes the DOM structure by adding/removing elements	@if, @for, *ngIf, *ngFor
Component	A directive with a template	Every @Component is technically a directive

Custom directives you write are almost always attribute directives — structural directives are rare and complex.

Core building blocks:

```
@Directive({ selector: '[appMyDirective]' }) // attaches to elements with this attribute
class MyDirective {
  @HostBinding('class.active') // binds to a property of the host element
  @HostListener('click') // listens to an event on the host element
  input() // receives configuration from the template
  ElementRef // direct reference to the host DOM element
  Renderer2 // safe, platform-agnostic DOM manipulation
}
```

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 17 — TypeScript Decorators: @Directive, @HostBinding, @HostListener are all decorators — understanding that decorators attach metadata to classes is essential.
- Topic 27 — Services & Dependency Injection: ElementRef, Renderer2, and any custom service can be injected using inject() — directives are full DI participants.
- Topic 26 — input() & output() in depth: Directives accept input() signals for configuration exactly as components do — the same binding syntax [appHighlight]="color" works identically.
- Topic 25 — Component Lifecycle Hooks: Directives have the same lifecycle hooks as components (ngOnInit, ngOnChanges, ngOnDestroy) — the hooks fire relative to the host element's lifecycle.
- Topic 30 — Built-in Directives: ngClass, ngStyle, and ngTemplateOutlet are Angular's own attribute directives — you've been using the pattern; now you'll build your own.

Bridging note: Every built-in directive you used in Topic 30 (ngClass, ngStyle) is implemented using exactly the same @Directive decorator, @HostBinding, and @HostListener pattern you'll learn here. You've been consuming this API since Topic 30 — now you're on the producing side.

Single most-direct extension of: Topic 30 — Built-in Directives.

Section 3 — Code Example

Plain TS — The core pattern in isolation

```

// A directive is a class that:
// 1. Knows which element it is attached to (ElementRef)
// 2. Can react to events on that element (@HostListener)
// 3. Can change properties of that element (@HostBinding)
// 4. Can accept configuration from the template (input())

class HighlightBehaviour {
  private element: HTMLElement;
  private color: string = 'yellow';

  constructor(el: HTMLElement, color: string) {
    this.element = el;
    this.color = color;
  }

  onMouseEnter() {
    this.element.style.backgroundColor = this.color;
  }

  onMouseLeave() {
    this.element.style.backgroundColor = '';
  }
}
// Angular's @Directive does this wiring automatically
// @HostListener replaces manual addEventListener
// @HostBinding replaces manual style assignment

```

Real Angular — Highlight directive

```
// directives/highlight.directive.ts
import { Directive, HostBinding, HostListener, input, effect } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true,
})
export class HighlightDirective {
  appHighlight = input<string>('yellow');
  defaultColor = input<string>('transparent');

  @HostBinding('style.backgroundColor') backgroundColor = '';

  constructor() {
    // effect() reacts to signal changes – replaces ngOnChanges for input syncing
    effect(() => {
      this.backgroundColor = this.defaultColor();
    });
  }

  @HostListener('mouseenter')
  onMouseEnter(): void {
    this.backgroundColor = this.appHighlight();
  }

  @HostListener('mouseleave')
  onMouseLeave(): void {
    this.backgroundColor = this.defaultColor();
  }
}
```

```
@Component({
  selector: 'app-product-card',
  standalone: true,
  imports: [HighlightDirective],
  template: `
    <div appHighlight="lightblue">Hover over me</div>
    <div [appHighlight]="selectedColor()" defaultColor="lightyellow">Dynamic</div>
    <p appHighlight>Default yellow highlight</p>
  `,
})
export class ProductCardComponent {
  selectedColor = signal('lightgreen');
}
```

Real Angular — Permission directive using Renderer2

```

@Directive({
  selector: '[appHasPermission]',
  standalone: true,
})
export class HasPermissionDirective {
  appHasPermission = input.required<string>();

  private el = inject(ElementRef);
  private renderer = inject(Renderer2);
  private auth = inject(AuthService);

  constructor() {
    effect(() => {
      if (!this.auth.hasPermission(this.appHasPermission())) {
        this.renderer.setStyle(this.el.nativeElement, 'display', 'none');
      }
    });
  }
}

// <button appHasPermission="admin:write" (click)="deleteUser()">Delete User</button>

```

Real Angular — Click-outside directive

```

@Directive({
  selector: '[appClickOutside]',
  standalone: true,
})
export class ClickOutsideDirective {
  // output() replaces @Output() clickOutside = new EventEmitter<void>()
  clickOutside = output<void>();
  private el = inject(ElementRef);

  @HostListener('document:click', ['$event.target'])
  onDocumentClick(target: HTMLElement): void {
    const isInside = this.el.nativeElement.contains(target);
    if (!isInside) { this.clickOutside.emit(); }
  }
}

// <div appClickOutside (clickOutside)="closeDropdown()">...</div>

```

Section 4 — Before & After Refactor

Before — DOM behaviour duplicated in every component

```
// product-card.component.ts
@Component({ ... })
export class ProductCardComponent {
  private el = inject(ElementRef);

  @HostListener('mouseenter')
  onEnter() { this.el.nativeElement.style.backgroundColor = 'lightyellow'; }

  @HostListener('mouseleave')
  onLeave() { this.el.nativeElement.style.backgroundColor = ''; }
}

// user-card.component.ts — SAME code duplicated
@Component({ ... })
export class UserCardComponent {
  private el = inject(ElementRef);

  @HostListener('mouseenter')
  onEnter() { this.el.nativeElement.style.backgroundColor = 'lightyellow'; }
  // ...identical code again
}
```

What breaks: Highlight behaviour is coupled to specific components. To add it to a third component, copy-paste again. To change the hover colour, update every component individually. This violates the single responsibility principle.

After — Behaviour extracted into a reusable directive

```
@Directive({ selector: '[appHighlight]', standalone: true })
export class HighlightDirective {
  appHighlight = input('lightyellow'); // input() signal, not @Input()

  @HostBinding('style.backgroundColor') bg = '';

  @HostListener('mouseenter') onEnter() { this.bg = this.appHighlight(); }
  @HostListener('mouseleave') onLeave() { this.bg = ''; }
}

// Apply to any element in any component:
// <app-product-card appHighlight="lightyellow" />
// <app-user-card appHighlight="lightblue" />
// <button appHighlight>Hover me</button>
```

Why this refactor matters: Any element in any component gains the behaviour by adding the attribute. Change the directive once — every element using it updates. This is the open/closed principle applied to Angular templates.

Section 5 — Common Mistakes

Mistake 1 — Manipulating the DOM directly via `ElementRef.nativeElement` instead of `Renderer2`

The mistake	<code>this.el.nativeElement.style.color = 'red'</code> directly in a directive
-------------	--

Why developers make it	It's shorter, familiar from vanilla JS, and works correctly in a browser
How to avoid it	Always use <code>Renderer2</code> : <code>this.renderer.setStyle(this.el.nativeElement, 'color', 'red')</code>
Angular consequence	Direct DOM manipulation breaks Angular Universal (SSR), Web Worker rendering, and test environments where <code>nativeElement</code> may not be a real DOM node

Mistake 2 — Using `@Input()` decorator instead of `input()` signal

The mistake	<code>@Input() appHighlight: string = 'yellow';</code>
Why developers make it	<code>@Input()</code> is the pattern most tutorials and existing codebases still show
How to avoid it	Use <code>appHighlight = input<string>('yellow');</code> — <code>input()</code> is the signal-based standard
Angular consequence	<code>@Input()</code> loses automatic change propagation into effects/computed signals; you need <code>ngOnChanges</code> manually instead of <code>effect()</code> reacting natively

Mistake 3 — Forgetting to add the directive to the component's imports array

The mistake	Creating a standalone directive but not importing it in the consuming component
Why developers make it	NgModule apps use declarations — muscle memory carries over
How to avoid it	Treat standalone directives like standalone components: they must be in imports: <code>[MyDirective]</code>
Angular consequence	Angular silently ignores the attribute — no error, no warning. The directive never instantiates

Mistake 4 — Using `@HostBinding` to bind to `innerHTML` for content projection

The mistake	<code>@HostBinding('innerHTML') content = 'Hello'</code>
Why developers make it	Looks like a quick way to add content to the host element from a directive

How to avoid it	If you need to add a template, you need a component (with ng-content), not a directive
Angular consequence	Bypasses Angular's change detection and template compilation. Event bindings don't work and XSS vulnerabilities are introduced if user data is involved

Section 6 — Do's and Don'ts

DO	DON'T
Use Renderer2 for all DOM manipulation inside directives	Use ElementRef.nativeElement directly for style/attribute changes
Use input() / input.required() signals for directive configuration	Use the @Input() decorator — signals give reactive change propagation @Input() doesn't
Use output() for events emitted from directives	Use @Output() + EventEmitter — output() is the signal-aligned replacement
Use @HostBinding to bind to host element properties declaratively	Manually set nativeElement.className or nativeElement.style in event handlers
Use @HostListener('document:click') for global event listeners	Add window.addEventListener manually — it leaks because Angular won't clean it up
Add the directive to the component imports array (standalone)	Assume directives auto-discover themselves
Use ngOnDestroy to clean up any subscriptions inside the directive	Forget that directives have their own lifecycle — subscriptions in directives leak

Section 7 — Interview Prep

Question 1 — Mid: "What is the difference between a component and a directive in Angular?"

Strong answer covers:

- Components have a template; directives do not
- Both are technically directives — @Component extends @Directive
- Attribute directives add behaviour/appearance to existing elements; structural directives change DOM structure
- When to choose each: own HTML structure -> component; modify existing element -> directive
- Both participate in DI, both have lifecycle hooks, both use input() / output()

Weak answer misses: That @Component is a directive with a template — treating them as completely different concepts rather than a specialisation relationship.

Question 2 — Mid: "What is Renderer2 and why should you use it instead of accessing nativeElement directly?"

Strong answer covers:

- Renderer2 is Angular's platform-agnostic DOM abstraction layer
- Direct nativeElement access couples code to the browser DOM — breaks in Angular Universal (SSR), Web Workers, and test environments
- Renderer2 methods: `setStyle`, `removeStyle`, `addClass`, `removeClass`, `setAttribute`, `removeAttribute`, `createElement`, `appendChild`
- Angular swaps the Renderer2 implementation based on the rendering environment
- Security benefit: Renderer2 routes through Angular's sanitisation pipeline; raw `innerHTML` does not

Weak answer misses: The SSR/platform-agnostic angle — just saying "it's the Angular way" without explaining why.

Question 3 — Mid-Senior: "How would you build a directive that listens to clicks anywhere on the document to close a dropdown, while ensuring it doesn't close on the click that opened it?"

Strong answer covers:

- `@HostListener('document:click', ['$event.target'])` for the global listener
- `ElementRef.nativeElement.contains(target)` to distinguish inside vs outside clicks
- The timing problem: the open-click event bubbles to document and immediately triggers the close listener
- Solution options: `stopPropagation` on the trigger button, or a one-tick delay using `setTimeout`
- `clickOutside = output<void>()` to communicate the event to the parent
- Cleanup is automatic — `@HostListener` on document is removed when the directive destroys

Weak answer misses: The timing/same-click problem — describing the basic pattern without addressing why it immediately closes on open.

Section 8 — Quick Reference Card

3-line definition:

A custom directive attaches behaviour to an existing host element via a CSS attribute selector. It has no template — it enhances the element it's placed on. `@HostBinding` changes how the element looks; `@HostListener` reacts to what happens to it; `input()` configures what it does.

Syntax at a glance:

Decorator/Function	What It Does	Example
<code>@Directive({ selector })</code>	Marks class as directive, sets CSS selector	selector: '[appHighlight]'
<code>@HostBinding('prop')</code>	Syncs directive property to host element property	<code>@HostBinding('class.active')</code> <code>isActive = false</code>

@HostListener('event')	Subscribes directive method to host DOM event	@HostListener('click') onClick()
input() / input.required()	Receives value from template binding as a signal	color = input<string>('yellow')
output()	Emits an event to the parent	clickOutside = output<void>()
ElementRef	Reference to the raw host DOM element	inject(ElementRef)
Renderer2	Platform-safe DOM manipulation API	renderer.setStyle(el, 'color', 'red')

When to use / when not to use:

Use a directive when	Use a component instead when
Same behaviour needed across multiple element types	The feature needs its own HTML structure
No template of its own is needed	Content projection is involved
Modifying appearance/behaviour of an existing element	The feature is self-contained with its own layout

The one rule that matters most:

Use `Renderer2` for all DOM manipulation inside directives — direct `nativeElement` access breaks Angular Universal (SSR) and test environments with no error in the browser.

State Management Patterns (Service with Signal / BehaviorSubject)

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The individual pieces (signals, BehaviorSubject, services) are already familiar from earlier topics; the difficulty here is architectural — understanding when to use which pattern, how to structure a store service correctly, and why naive approaches cause bugs at scale.

Angular Relevance: Critical — Every non-trivial Angular app needs a shared state strategy; this is the pattern you reach for before installing NgRx or Akita, and it covers 80% of real-world Angular state requirements with zero third-party dependencies.

Section 1 — Explanation

The Whiteboard Analogy

Imagine a shared office whiteboard that every team member can read at any time. When someone updates a number on the whiteboard, everyone in the room immediately sees the new value — they don't need to be told, they just look up and it's already changed.

A state management service is that whiteboard. Instead of each component holding its own local copy of data (their own private notepad), they all read from and write to one central service. The service owns the data. Components are just displays and input devices.

Technical Explanation

Both patterns follow the same architectural shape:

```
Private mutable state  -> Public read-only projection  -> Components read & display
Public write methods   -> Components call to update
```

Corrected framing: In current Angular (v17+ with zoneless change detection on the roadmap), the signal store is the default choice for new state — not one of two roughly-equal options. Signals are pull-based and integrate directly with Angular's change detection without requiring Zone.js. BehaviorSubject is push-based and historically relied on Zone.js. As Angular moves toward zoneless operation via `provideExperimentalZonelessChangeDetection()`, BehaviorSubject-based state that isn't bridged through `toSignal()` or manually marked for `check` stops updating the view reliably. Signal stores have no such dependency.

	Signal Store (default)	BehaviorSubject Store (special case only)
State primitive	<code>signal<T>(initialValue)</code>	<code>new BehaviorSubject<T>(initialValue)</code>
Public read API	<code>readonly count = this.#count.asReadonly()</code>	<code>readonly count\$ = this.#count.asObservable()</code>

Write API	<code>this.#count.set(n) / .update(fn)</code>	<code>this.#count.next(newValue)</code>
Template consumption	<code>{{ store.count() }}</code>	<code>{{ store.count\$ async }}</code>
Subscription cleanup	None — signals are not subscriptions	Required — <code>takeUntilDestroyed()</code> or <code>async pipe</code>
Zone.js dependency	None — works identically <code>zoneless</code> or <code>zone-based</code>	Historically <code>Zone.js</code> -triggered; silently stops updating under <code>zoneless</code> unless bridged via <code>toSignal()</code>
Recommended default	Always start here for new state	Only when store logic is genuinely pipeline-shaped

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 23 — Angular Signals: the signal store pattern is a direct architectural application of `signal()`, `computed()`, and `effect()`
- Topic 20 — RxJS Subscription Management: the `BehaviorSubject` pattern requires proper subscription cleanup
- Topic 21 — `takeUntilDestroyed` & `DestroyRef`: `BehaviorSubject` stores must use `takeUntilDestroyed()` to prevent memory leaks
- Topic 22 — `async Pipe`: `BehaviorSubject`-based stores are consumed in templates via the `async pipe`
- Topic 24 — `OnPush` Change Detection: signal-based stores work perfectly with `OnPush` because signal reads are tracked
- Topic 27 — Services & Dependency Injection: the store is a service; `providedIn: 'root'` vs feature-level provision determines state scope

Bridging note: In Topic 23 you learned that `signal()` holds a value and notifies readers when it changes. This topic takes that knowledge and applies it architecturally: the signal lives in a service (not a component), making it shared across the entire app. The component's job shrinks from "hold state + transform it + display it" to just "display it and call write methods."

Most directly extends: Topic 23 — Angular Signals (the signal store is signals applied at architectural scale).

Section 3 — Code Example

Plain TS — The shape of a store

```

class CounterStore {
  #count = 0;
  get count() { return this.#count; }

  increment() { this.#count++; }
  decrement() { this.#count--; }
  reset()     { this.#count = 0; }
}

const store = new CounterStore();
store.increment();
console.log(store.count); // 1
// store.#count = 999;    // compile error -- private field

```

Real Angular — Signal-based store

```

// cart.store.ts
@Inject({ providedIn: 'root' })
export class CartStore {
  readonly #items      = signal<CartItem[]>([]);
  readonly #isLoading = signal(false);

  readonly items      = this.#items.asReadOnly();
  readonly isLoading = this.#isLoading.asReadOnly();

  readonly itemCount = computed(() => this.#items().reduce((sum, i) => sum + i.quantity, 0));
  readonly totalPrice = computed(() => this.#items().reduce((sum, i) => sum + i.price * i.quantity, 0));
  readonly isEmpty    = computed(() => this.#items().length === 0);

  addItem(item: CartItem): void {
    this.#items.update(current => {
      const exists = current.find(i => i.id === item.id);
      if (exists) {
        return current.map(i => i.id === item.id
          ? { ...i, quantity: i.quantity + 1 } : i);
      }
      return [...current, item];
    });
  }

  removeItem(id: number): void {
    this.#items.update(current => current.filter(i => i.id !== id));
  }

  clearCart(): void { this.#items.set([]); }
}

```

```
// cart-icon.component.ts
@Component({
  selector: 'app-cart-icon',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <button>
      Cart
      @if (store.itemCount() > 0) {
        <span class="badge">{{ store.itemCount() }}</span>
      }
    </button>
  `
})
export class CartIconComponent {
  protected store = inject(CartStore);
}
```

Real Angular — BehaviorSubject-based store

```
// auth.store.ts
@Injectable({ providedIn: 'root' })
export class AuthStore {
  readonly #state = new BehaviorSubject<AuthState>(INITIAL_STATE);

  readonly state$ = this.#state.asObservable();
  readonly user$ = this.state$.pipe(map(s => s.user));
  readonly isLoggedIn$ = this.state$.pipe(map(s => s.token !== null));

  #patch(partial: Partial<AuthState>): void {
    this.#state.next({ ...this.#state.getValue(), ...partial });
  }

  loginSuccess(user: User, token: string): void {
    this.#patch({ user, token, isLoading: false });
  }

  logout(): void { this.#state.next(INITIAL_STATE); }

  getToken(): string | null { return this.#state.getValue().token; }
}
```

Section 4 — Before & After Refactor

Before — State scattered across components

```
// WRONG: every component manages its own copy
@Component({ template: `<span>{{ user?.name }}</span>` })
export class HeaderComponent implements OnInit {
  user: User | null = null;
  ngOnInit() {
    this.authService.getUser().subscribe(u => this.user = u);
    // HTTP call fires AGAIN every time HeaderComponent initialises
  }
}
// Dashboard has the same code -- three copies, three HTTP calls, three stale sources
```

What breaks: Three separate HTTP requests for the same data. If the user updates their profile, only the component that triggered the update reflects the change. Every component carries subscription cleanup boilerplate.

After — Single source of truth in a store service

```
@Injectable({ providedIn: 'root' })
export class UserStore {
  readonly #user = signal<User | null>(null);
  readonly user = this.#user.asReadonly();
  private http = inject(HttpClient);

  loadUser(): void {
    this.http.get<User>('/api/me').subscribe(u => this.#user.set(u));
  }
}

// HeaderComponent -- zero HTTP calls, zero subscription management
@Component({ template: `<span>{{ store.user()?.name }}</span>` })
export class HeaderComponent {
  protected store = inject(UserStore);
}

// DashboardComponent -- same store, automatically stays in sync
@Component({ template: `<p>Hello {{ store.user()?.name }}</p>` })
export class DashboardComponent {
  protected store = inject(UserStore);
}
```

Section 5 — Common Mistakes

Mistake 1 — Exposing the mutable signal or BehaviorSubject directly

The mistake	items = signal<CartItem[]>([]) as a public property
Why developers make it	Less code. Direct access feels simpler
How to avoid it	Always use #privateSignal + public readonly = this.#privateSignal.asReadonly()
Angular consequence	State mutations bypass the store's logic, making it impossible to add side effects (analytics, persistence, validation) later

Mistake 2 — Creating a new store instance per component

The mistake	Listing providers: [CartStore] in a @Component's metadata
Why developers make it	Developers copy-paste a service provider not realising it creates a scoped instance

How to avoid it	State management services must use <code>providedIn: 'root'</code> . Never list a store in a component's providers: []
Angular consequence	Each component gets a fresh, empty store instance. <code>CartItemComponent</code> shows 0 items while <code>ProductListComponent</code> shows 3 -- no error, just silently disconnected state

Mistake 3 — Subscribing to a BehaviorSubject store without cleanup

The mistake	<pre>ngOnInit() { this.authStore.user\$.subscribe(user => { this.currentUser = user; }); }</pre>
Why developers make it	The async pipe isn't always applicable, so developers subscribe manually and forget cleanup
How to avoid it	Use <code>takeUntilDestroyed(this.destroyRef)</code> on every manual BehaviorSubject subscription
Angular consequence	The destroyed component's callback continues firing. In long sessions this accumulates -- the classic Angular memory leak

Mistake 4 — Assuming BehaviorSubject stores work correctly under zoneless Angular

The mistake	Using BehaviorSubject stores as the default without verifying Zone.js dependency
Why developers make it	BehaviorSubject stores work in Zone.js-based apps, so developers don't notice the coupling
How to avoid it	Default to signal stores for all new state. Bridge BehaviorSubject output through <code>toSignal()</code> before it reaches the template
Angular consequence	Under <code>provideExperimentalZonelessChangeDetection()</code> , BehaviorSubject emissions may not trigger view updates -- no error, just a silently stale UI

Section 6 — Do's and Don'ts

DO	DON'T
Use private fields (<code>#state</code>) and expose only <code>asReadonly()</code> signals or <code>.asObservable()</code> streams	Expose mutable signals or BehaviorSubjects publicly

Default to signal stores for all new state in Angular v17+	Treat BehaviorSubject as a co-equal default -- it carries Zone.js dependency that signals don't
Put the store in providedIn: 'root' for app-wide shared state	Add the store to a component's providers: []
Use computed() for all derived state in signal stores	Recompute derived values inside components
Use update(fn) for signal mutations that depend on current value	Call set() with a manually retrieved current value -- race condition risk
Consume BehaviorSubject stores via async pipe or bridge via toSignal()	Manually subscribe without takeUntilDestroyed

Section 7 — Interview Prep

Question 1 — Mid: "How would you share state between two sibling components without using input()/output() chains?"

Strong answer covers: lift state into a shared providedIn: 'root' service; both components inject the same instance; writing to the store from one component automatically updates all other consumers; mention the alternative of a parent component as mediator but note it doesn't scale; signal stores need no subscription management; BehaviorSubject stores need async pipe or takeUntilDestroyed.

Weak answers miss: The providedIn: 'root' requirement -- many candidates say "put it in a service" without knowing component-level providers create isolated instances.

Question 2 — Mid: "Why should you expose asReadonly() from a signal store instead of the raw signal?"

Strong answer covers: asReadonly() returns a Signal<T> with no set() or update() methods; without it, any component can call store.items.set([]) directly bypassing all store logic; bypassed logic means no validation, no derived state updates, no analytics events; same principle as private in OOP; in a large team, "please don't call .set() directly" is not enforceable -- types are.

Weak answers miss: The consequence side -- candidates know what asReadonly() does but not what breaks in a real codebase.

Question 3 — Senior: "When would you choose a BehaviorSubject-based store over a signal-based store in a modern Angular application?"

Strong answer covers: when store logic involves complex async pipelines (switchMap, combineLatest, conditional stream branching); when synchronous snapshots are needed by non-reactive consumers like HTTP interceptors (getValue()); when the team is already heavily invested in RxJS; acknowledge signals are preferred for 80% of cases in v17+; flag the Zone.js dependency: BehaviorSubject stores can silently stop updating views under zoneless unless bridged via toSignal(); the two patterns can coexist.

Weak answers miss: The Zone.js dependency angle and the interceptor/synchronous-consumer scenario.

Section 8 — Quick Reference Card

3-line definition:

A state management service is the single source of truth — private mutable state, public read-only projection, public write methods. Signal stores are the default for all new Angular state: no subscriptions, no Zone.js dependency, perfect OnPush integration. BehaviorSubject stores are the exception — justified only when the store's internal logic is genuinely pipeline-shaped or its consumers are RxJS-native.

Decision	Signal Store	BehaviorSubject Store
Angular version	v17+ -- the default for all new state	Any version -- special case only
Template consumption	<code>{{ store.value() }}</code>	<code>{{ store.value\$ async }}</code>
Synchronous snapshot	<code>store.value()</code>	<code>subject.getValue()</code>
Derived state	<code>computed(() => ...)</code>	<code>.pipe(map(...))</code>
Subscription cleanup	Not needed	<code>takeUntilDestroyed()</code> or <code>async pipe</code>
Zone.js dependency	None	Historically Zone.js-triggered; bridge via <code>toSignal()</code> for zoneless safety
New projects default	Always prefer signals	Only for genuinely pipeline-shaped internal logic

The one rule that matters most:

Never expose a writable signal or BehaviorSubject publicly -- `asReadonly()` and `asObservable()` are not optional; they are the type-level enforcement of your store's contract.

RxJS Operators in Depth

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — Each operator is individually learnable, but the real difficulty is the judgment layer: knowing which operator to reach for in a given async scenario, understanding the cancellation and concurrency semantics of each, and recognising the bugs that appear when the wrong operator is used — bugs that are silent, intermittent, and data-corrupting.

Angular Relevance: Critical — These six operators appear in virtually every Angular app that touches HTTP, user input, or real-time data; choosing the wrong one between `switchMap` and `mergeMap` in an HTTP search handler is one of the most common sources of race conditions in Angular production apps.

Section 1 — Explanation

The Food Delivery App Analogy

Imagine you're managing a food delivery dispatch system. A customer can place an order at any time. How you handle a new order arriving while a previous order is still being prepared defines which operator you are:

- `switchMap` — "Cancel the current order, start the new one immediately." Best for: search queries, autocomplete — only the latest request matters.
- `mergeMap` — "Run all orders simultaneously, no limit." Best for: independent parallel actions (liking multiple posts).
- `concatMap` — "Queue orders strictly. Finish one completely before starting the next." Best for: sequential operations where order matters.
- `forkJoin` — "Wait until ALL orders are done, then serve everything at once." Best for: loading multiple independent API resources before rendering a page.
- `combineLatest` — "Every time ANY order updates, re-serve the full current state of all orders." Best for: combining multiple live streams.
- `debounceTime` — "Wait until the customer stops changing their order for X milliseconds before sending it to the kitchen." Best for: search inputs, form validation.

The core distinction

Operator	Inner observable behaviour	Cancels previous?	Order guaranteed?
<code>switchMap</code>	Replaces -- cancels previous	Yes	N/A -- only latest
<code>mergeMap</code>	Merges -- all run concurrently	No	No
<code>concatMap</code>	Queues -- waits for each to complete	No	Yes

forkJoin	Combines -- waits for all to complete once	N/A	Yes -- emits together
combineLatest	Combines -- emits on any source change	N/A	Latest from each
debounceTime	Delays -- suppresses rapid emissions	Implicitly	N/A

Section 2 — Connection to Prior Concepts

- Topic 20 — RxJS Subscription Management: every operator produces a new observable that must be subscribed to and cleaned up
- Topic 21 — takeUntilDestroyed & DestroyRef: all component-level subscriptions need takeUntilDestroyed()
- Topic 36 — HttpClient & API Integration: HttpClient returns cold observables; switchMap, mergeMap, concatMap, and forkJoin are the primary composition tools
- Topic 37 — HTTP Interceptors: interceptors use switchMap internally for token refresh patterns
- Topic 44 — State Management Patterns: BehaviorSubject stores use combineLatest and switchMap in their pipeline logic
- Topic 45 — Performance Optimisation: debounceTime is the companion to pure pipe filtering

Most directly extends: Topic 36 — HttpClient & API Integration.

Section 3 — Code Example

switchMap — Cancel and replace

```
// The search autocomplete pattern
this.results$ = this.query$.pipe(
  debounceTime(300),           // wait 300ms after last keystroke
  distinctUntilChanged(),      // skip if query hasn't changed
  filter(q => q.length >= 2),
  switchMap(query =>           // cancel previous HTTP call, start new one
    this.http.get<SearchResult[]>('/api/search', { params: { q: query } }).pipe(
      catchError(() => of([]))
    )
  )
);
// WHY switchMap: user types 'a'-'>'an'-'>'ang'-'>'...-'>'angular'
// Without switchMap (mergeMap): 6 requests fire, old 'a' response can arrive AFTER
// 'angular' response -- user sees 200 wrong results silently
```

mergeMap — Run all concurrently

```
// Liking multiple posts independently -- order doesn't matter, none should cancel
this.likeAction$.pipe(
  mergeMap(postId =>
    this.http.post(`/api/posts/${postId}/like`, {}).pipe(
      catchError(err => { console.error(err); return EMPTY; })
    )
  ),
  takeUntilDestroyed(this.destroyRef)
).subscribe();
// WHY mergeMap: switchMap would cancel the post 1 like when post 2 is liked -- wrong
```

concatMap — Queue in order

```
uploadFiles(files: File[]): Observable<UploadResult[]> {
  return from(files).pipe(
    concatMap(file => {
      const formData = new FormData();
      formData.append('file', file);
      return this.http.post<UploadResult>('/api/upload', formData).pipe(
        catchError(err => of({ file: file.name, error: err.message }))
      );
    }),
    toArray()
  );
}
// WHY concatMap: server has concurrency limit; mergeMap would fire all simultaneously
```

forkJoin — Wait for all to complete

```
loadProductPage(productId: number): Observable<ProductPageData> {
  return forkJoin({
    product: this.http.get<Product>(`/api/products/${productId}`)
      .pipe(catchError(() => of(null))),
    reviews: this.http.get<Review[]>(`/api/products/${productId}/reviews`)
      .pipe(catchError(() => of([]))),
    related: this.http.get<Product[]>(`/api/products/${productId}/related`)
      .pipe(catchError(() => of([])))
  });
}
// CRITICAL: if ANY source errors, forkJoin errors -- always add per-source catchError
```

combineLatest — Emit on any source change

```
this.filteredUsers$ = combineLatest([
  this.userStore.users$,
  this.searchQuery$, // BehaviorSubject -- has initial value
  this.statusFilter$ // BehaviorSubject -- has initial value
]).pipe(
  debounceTime(100),
  map([users, query, status]) =>
    users
      .filter(u => !query || u.name.toLowerCase().includes(query.toLowerCase()))
      .filter(u => !status || u.status === status)
  );
// WHY combineLatest and not forkJoin:
// forkJoin waits for ALL sources to COMPLETE -- BehaviorSubjects never complete
```

tap() for state mutation -- the anti-pattern

A critical pattern to avoid: using tap() to mutate signal state inside an RxJS pipeline. Three specific failure modes:

- Failure 1: switchMap cancellation leaves loading stuck. If the user clears the input and switchMap cancels the last request, the "loading false" tap() never runs -- spinner stays forever.
- Failure 2: Replay and multicasting break the state. A late subscriber to a shareReplay(1) observable triggers the "loading false" tap without ever triggering "loading true."
- Failure 3: Untestability. Asserting isLoading transitions requires subscribing to the full pipeline in tests.

The correct pattern: write signal state from named, explicit methods co-located with the action that caused the transition -- never from tap() embedded in value-transformation chains.

```
// WRONG: tap() for state mutation
results$ = this.query$.pipe(
  tap(() => this.isLoading.set(true)), // fires on every emission
  switchMap(q => this.http.get('/api/search', { params: { q } })),
  tap(() => this.isLoading.set(false)) // doesn't fire if switchMap cancels
);

// CORRECT: explicit state transitions
switchMap(q => {
  this.isLoading.set(true); // co-located with the action that caused it
  this.hasError.set(false);
  return this.http.get('/api/search', { params: { q } }).pipe(
    tap({ complete: () => this.isLoading.set(false) }),
    catchError(() => {
      this.isLoading.set(false);
      this.hasError.set(true);
      return of([]);
    })
  );
});
```

Section 4 — Before & After Refactor

Before — Wrong operator causes a race condition

```
ngOnInit() {
  this.results$ = this.query$.pipe(
    mergeMap(query => // WRONG operator
      this.http.get<Result[]>('/api/search', { params: { q: query } })
    )
  );
}
// Scenario: User types 'a' (slow, 800ms) then 'an' (fast, 100ms)
// 'an' response arrives first -- correct results shown
// 'a' response arrives 700ms later -- WRONG results now shown
// No error, no warning -- silent data corruption
```

After — switchMap eliminates the race condition

```

ngOnInit() {
  this.results$ = this.query$.pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(query => // correct operator
      this.http.get<Result[]>('/api/search', { params: { q: query } })).pipe(
        catchError(() => of([]))
      )
    )
  );
}
// switchMap with HttpClient ACTUALLY cancels the in-flight HTTP request
// via AbortController -- saves both bandwidth AND server CPU

```

Section 5 — Common Mistakes

Mistake 1 — Using switchMap for write operations (POST/PUT/DELETE)

The mistake	this.saveAction\$.pipe(switchMap(data => this.http.put('/api/record', data)))
Why developers make it	switchMap is the "correct" HTTP operator -- learned for search and applied everywhere
How to avoid it	Use concatMap for sequential saves, exhaustMap for "ignore new saves while one is in progress"
Angular consequence	In a rapid double-click, the first PUT is cancelled mid-flight. Data may be partially written or fully lost

Mistake 2 — Using forkJoin on observables that may not complete

The mistake	forkJoin([this.http.get('/api/data'), this.userStore.user\$]) where user\$ is a BehaviorSubject
Why developers make it	forkJoin is described as "combine multiple observables"
How to avoid it	forkJoin requires every source to COMPLETE. BehaviorSubject never completes. Use combineLatest for live streams
Angular consequence	The subscription silently hangs forever -- subscribe() never fires, no error

Mistake 3 — Nesting subscribe() inside subscribe()

The mistake	this.route.paramMap.subscribe(params => { this.http.get(...).subscribe(data => ...) })
-------------	--

Why developers make it	Mirrors Promise chaining with <code>.then()</code> -- familiar and intuitive at first
How to avoid it	The inner observable should be a flattening operator on the outer observable
Angular consequence	Inner subscription has no connection to outer's lifecycle. <code>takeUntilDestroyed()</code> on the outer fires but the inner leaks

Mistake 4 — Using `tap()` for state mutation inside RxJS pipelines

The mistake	<code>tap() => this.isLoading.set(true))</code> before <code>switchMap</code> , <code>tap() => this.isLoading.set(false))</code> after
Why developers make it	Reads linearly -- "when a value arrives, set loading true; when a result arrives, set loading false"
How to avoid it	Write signal state from explicit, named methods -- never from <code>tap()</code> embedded in value-transformation chains
Angular consequence	If <code>switchMap</code> cancels the inner observable, the "loading false" <code>tap()</code> never runs -- spinner stays forever. Late subscribers to <code>shareReplay(1)</code> trigger "loading false" without "loading true"

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>switchMap</code> for user-triggered search/autocomplete HTTP calls	Use <code>switchMap</code> for POST/PUT/DELETE -- it cancels in-flight requests
Use <code>concatMap</code> for sequential write operations	Use <code>mergeMap</code> for writes that must not overlap
Use <code>forkJoin</code> only on observables guaranteed to complete (HttpClient calls)	Use <code>forkJoin</code> with <code>BehaviorSubject</code> or <code>Subject</code> -- they never complete
Add <code>catchError</code> to each individual source inside <code>forkJoin</code>	Let one failing <code>forkJoin</code> source kill all other completed results
Use <code>combineLatest</code> with <code>BehaviorSubject</code> or <code>startWith()</code> to guarantee initial emission	Use <code>combineLatest</code> without ensuring all sources have emitted at least once
Write signal state from named, explicit methods	Use <code>tap()</code> for state mutations with a defined start/end
Pair flattening operators with <code>takeUntilDestroyed()</code>	Nest <code>subscribe()</code> inside <code>subscribe()</code>

Section 7 — Interview Prep

Question 1 — Mid/Senior: "What is the difference between switchMap, mergeMap, and concatMap, and when would you use each?"

Strong answer: switchMap cancels the previous inner observable -- use for search/autocomplete where only the latest result matters. mergeMap runs all concurrently -- use for independent parallel actions. concatMap queues inner observables one at a time -- use for sequential write operations. Critical rule: never use switchMap for POST/PUT/DELETE.

Weak answers miss: The POST/PUT/DELETE rule for switchMap.

Question 2 — Mid: "Why might a forkJoin never emit, and how would you debug it?"

Strong answer: forkJoin requires all source observables to COMPLETE -- not just emit. BehaviorSubject, Subject, combineLatest, and WebSocket streams never complete. Debug with tap({ complete: () => console.log('completed') }) on each source. Fix: use take(1) on non-completing sources, or switch to combineLatest.

Weak answers miss: The completion requirement.

Question 3 — Mid/Senior: "A developer uses combineLatest but the template never renders. What are the two most likely causes?"

Strong answer: Cause 1 -- one source uses Subject instead of BehaviorSubject; combineLatest requires every source to have emitted at least once; fix with BehaviorSubject or startWith(). Cause 2 -- the observable isn't subscribed to; async pipe missing in template.

Weak answers miss: The "all sources must have emitted at least once" requirement.

Section 8 — Quick Reference Card

Operator decision tree:

Question	Answer	Operator
Is it a search / autocomplete?	Only latest result matters	switchMap
Is it a write operation (POST/PUT)?	Must not cancel mid-flight	concatMap or exhaustMap
Are operations independent and concurrent?	All should complete	mergeMap
Do you need all results at once before rendering?	One-shot parallel load	forkJoin
Do you need the latest from multiple live streams?	Combine live streams	combineLatest
Are events too rapid?	Wait for quiet period	debounceTime
Need to set loading state during async?	Named signal writes, not tap()	Explicit signal.set()

The one rule that matters most:

Never use `switchMap` for POST/PUT/DELETE operations -- it cancels in-flight write requests on double-click or rapid re-trigger, silently corrupting or losing data.

Feature-based Folder Structure

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner/Intermediate — The concepts are straightforward; the difficulty is discipline. There's no compiler enforcement — a poorly structured project works identically to a well-structured one until the team grows or the codebase scales, at which point the cost of the wrong structure compounds rapidly.

Angular Relevance: High — Every Angular app needs a folder structure decision on day one. Feature-based structure is the Angular style guide recommendation and the de facto standard in production codebases; getting it wrong means expensive refactors later.

Section 1 — Explanation

The Department Store Analogy

Imagine a department store. One approach organises by item type: all shirts on floor 1, all trousers on floor 2, all shoes on floor 3 — regardless of brand or use. This is type-based structure: all components in one folder, all services in another, all models in another.

The other approach organises by department: the men's section has its own shirts, trousers, shoes, and accessories together in one area. Each department is self-contained. This is feature-based structure: everything belonging to a feature lives together.

The department store model wins at scale because:

- A new product line (feature) means adding a new department — existing departments are untouched
- A developer works entirely within one department — no cross-floor trips
- Removing a department means removing one section — not hunting across all floors for scattered items

Technical Explanation

Feature-based structure groups files by what they belong to (a feature, a domain, a page) rather than what they are (a component, a service, a model).

Type-based (avoid at scale):

```
src/app/  
  components/  
    header.component.ts  
    user-card.component.ts  
    product-list.component.ts  
  services/  
    auth.service.ts  
    user.service.ts  
  models/  
    user.model.ts
```

Feature-based (the standard):

```

src/app/
  features/
    auth/
      auth.component.ts
      auth.service.ts
      auth.routes.ts
      models/user.model.ts
    products/
      product-list.component.ts
      product-detail.component.ts
      product.service.ts
      product.routes.ts
      models/product.model.ts
  shared/
    ui/
      button.component.ts
      card.component.ts
      utils/date.utils.ts
      pipes/currency-format.pipe.ts
  core/
    auth.interceptor.ts
    error.interceptor.ts
    app.config.ts

```

Three top-level zones:

- features/ -- one folder per route/domain; self-contained; lazy-loaded
- shared/ -- reusable UI components, pipes, directives used across multiple features; no feature-specific logic
- core/ -- singleton services, interceptors, app-wide config; provided once at root level

Section 2 — Modern Angular Pattern (v17+)

In modern Angular with standalone components, feature-based structure maps directly onto lazy-loaded route chunks. Each feature folder owns its routes file, and the router loads the feature on demand.

```

// app.routes.ts
import { Routes } from '@angular/router';

export const appRoutes: Routes = [
  {
    path: 'dashboard',
    loadChildren: () =>
      import('./features/dashboard/dashboard.routes')
        .then(m => m.DASHBOARD_ROUTES)
  },
  {
    path: 'orders',
    loadChildren: () =>
      import('./features/orders/orders.routes')
        .then(m => m.ORDER_ROUTES)
  },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
];

```

```
// features/orders/orders.routes.ts
export const ORDER_ROUTES: Routes = [
  { path: '', component: OrderListComponent },
  { path: ':id', component: OrderDetailComponent }
];
```

Section 3 — Code Example

The placement rule

```
// Ask this question for every file you create:
// "If I deleted the entire [feature] folder, would this file break another feature?"
//
// YES -> it belongs in shared/ or core/
// NO -> it belongs inside the feature folder
//
// Examples:
// auth.service.ts used only in auth/ -> features/auth/services/auth.service.ts
// button.component.ts used in auth/, orders/, dashboard/ -> shared/ui/button/
// auth.interceptor.ts wires up at root level -> core/interceptors/
```

Full Angular example

```
// features/orders/services/order.service.ts
@Injectable({ providedIn: 'root' })
export class OrderService {
  private http = inject(HttpClient);

  readonly #orders = signal<Order[]>([]);
  readonly orders = this.#orders.asReadOnly();
  readonly total = computed(() => this.#orders().length);

  loadOrders(): void {
    this.http.get<Order[]>('/api/orders')
      .subscribe(orders => this.#orders.set(orders));
  }
}
```

```
// features/orders/order-list.component.ts
@Component({
  selector: 'app-order-list',
  standalone: true,
  imports: [ButtonComponent], // shared component -- not co-located
  template: `
    <h1>Orders ({{ service.total() }})</h1>
    @for (order of service.orders(); track order.id) {
      <div>{{ order.id }} -- {{ order.status }}</div>
    }
  `
})
export class OrderListComponent implements OnInit {
  protected service = inject(OrderService);
  ngOnInit() { this.service.loadOrders(); }
}
```

```
// shared/ui/button/button.component.ts
// Lives here because: used across orders/, auth/, dashboard/
// Has ZERO knowledge of any feature
@Component({
  selector: 'app-button',
  standalone: true,
  template: `
    <button [disabled]="disabled()" [class]="variant()">
      <ng-content />
    </button>
  `
})
export class ButtonComponent {
  disabled = input(false);
  variant = input<'primary' | 'secondary'>('primary');
}
```

Section 4 — Before & After Refactor

Before — Type-based structure

```
src/app/
  components/
    login.component.ts
    register.component.ts
    order-list.component.ts
    order-detail.component.ts
    dashboard.component.ts
    button.component.ts
  services/
    auth.service.ts
    order.service.ts
  models/
    user.model.ts
    order.model.ts
```

What breaks: Adding a new developer requires understanding the entire services/ folder to work on one feature. Deleting the orders feature requires hunting through components/, services/, models/ to find and remove scattered files. Lazy loading is impossible — all components live in one bundle.

After — Feature-based structure

```

src/app/
  features/
    auth/
      login.component.ts
      register.component.ts
      auth.routes.ts
      services/auth.service.ts
      guards/auth.guard.ts
      models/user.model.ts
    orders/
      order-list.component.ts
      order-detail.component.ts
      orders.routes.ts
      services/order.service.ts
      models/order.model.ts
    dashboard/
      dashboard.component.ts
      dashboard.routes.ts
      services/dashboard.service.ts
  shared/
    ui/button/button.component.ts

```

Why it matters: Deleting the orders feature = deleting one folder. Lazy loading = one loadChildren per feature. A developer working on orders never opens features/auth/. Scope is enforced by location.

Section 5 — Common Mistakes

Mistake 1 — Putting everything in shared/

The mistake	shared/services/order.service.ts -- using shared/ as a second components/ folder
Why developers make it	"It might be reused someday" -- over-engineering for hypothetical reuse
How to avoid it	shared/ is for things currently used by two or more features. Move it to shared/ when the second consumer appears
Angular consequence	shared/ becomes a dumping ground. Lazy loading loses its benefit -- shared feature logic gets bundled into the main chunk

Mistake 2 — Cross-feature imports

The mistake	features/orders/order-list.component.ts imports from features/auth/services/auth.service.ts
Why developers make it	It's the quickest path to the data needed

How to avoid it	Features must never import from other features. Shared state goes through core/ stores or shared/ services
Angular consequence	Features become coupled. Deleting or refactoring auth/ breaks orders/. Lazy loading can create circular chunk dependencies

Mistake 3 — One flat features/ folder with no sub-structure

The mistake	features/order-list.component.ts, features/auth.service.ts -- all feature files in one flat directory
Why developers make it	Starting small -- "I only have a few files, structure feels premature"
How to avoid it	One sub-folder per route/domain from the start. It costs nothing to create a folder
Angular consequence	The features/ folder becomes the same problem as the original components/ folder

Section 6 — Do's and Don'ts

DO	DON'T
One folder per feature/route in features/	Put feature-specific services or models in shared/ "just in case"
Move a file to shared/ only when a second feature needs it	Import directly from another feature's folder
Keep core/ for singleton infrastructure: interceptors, root-level config, app-wide stores	Put page components or feature services in core/
Co-locate the feature's routes file with the feature: orders/orders.routes.ts	Put all routes in one global routes file once features grow beyond 3-4 routes
Keep shared/ components purely presentational	Let shared/ components inject feature-specific services
Name feature folders after the domain/route	Name folders pages/, containers/, presentational/ -- type-based naming in disguise

Section 7 — Interview Prep

Question 1 — Mid: "How would you structure a large Angular application with 10+ features?"

Strong answer: features/ -- one folder per route/domain, self-contained, lazy-loaded; shared/ -- reusable UI with no feature knowledge; core/ -- singleton infrastructure: interceptors, app config, root stores; each feature owns its own routes file, services, models, and child components; the rule: "does any other feature currently use this?" -- yes -> shared/, no -> stay in feature; lazy loading via loadChildren in root routes.

Weak answers miss: The core/ vs shared/ distinction.

Question 2 — Mid/Senior: "A junior developer is importing a service from the auth feature directly into the orders feature component. What's the problem?"

Strong answer: Cross-feature imports create coupling -- deleting or refactoring auth/ breaks orders/. Lazy loading creates potential circular chunk dependencies. Fix: the shared state (current user, auth token) belongs in a core/ store or shared/ service. The auth feature writes to the store; the orders feature reads from it -- no direct feature-to-feature dependency. Enforce with ESLint rules.

Weak answers miss: The lazy loading bundle implication and the architectural fix.

Section 8 — Quick Reference Card

3-line definition:

Feature-based structure groups all files for a feature (components, services, models, routes, guards) in one folder per domain. shared/ holds components used by two or more features. core/ holds singleton infrastructure used app-wide. Location communicates ownership.

Zone	What goes here	Rule
features/<name>/	Components, services, models, routes, guards for one feature	Only this feature imports from here
shared/	UI components, pipes, directives used by 2+ features	No feature-specific logic
core/	Interceptors, app config, root stores, singleton services	Provided once at root; never feature-specific

The one rule that matters most:

Features must never import from other features -- shared state routes through core/ or shared/, never through direct cross-feature imports.

Testing Angular Components (TestBed, ComponentFixture, fakeAsync)

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — The APIs (TestBed, ComponentFixture, fakeAsync) are learnable, but the difficulty is architectural: knowing what to test vs. what to mock, understanding Angular's test change detection cycle (which differs from production), and correctly handling async operations in a synchronous test runner without creating flaky tests.

Angular Relevance: Critical — Component tests are the primary safety net for Angular UI logic; without them, refactoring components, upgrading Angular versions, or changing templates becomes a high-risk activity; every production Angular team requires component testing fluency.

Section 1 — Explanation

The Flight Simulator Analogy

A flight simulator lets pilots practise in a controlled environment that behaves like a real aircraft — without the real aircraft, real passengers, or real consequences.

TestBed is Angular's flight simulator. It creates a miniature Angular environment — a real (but sandboxed) Angular application — inside your test runner with real Angular change detection, real template compilation, real dependency injection, and controlled substitutes for external dependencies.

- **ComponentFixture** is your control panel — it gives you handles to trigger change detection, access the DOM, inspect the component instance, and observe outputs.
- **fakeAsync / tick()** is the simulator's time control — it lets you fast-forward through timers, debounce periods, and async operations without waiting for real time to pass.

Layer	Tool	What it does
Setup	<code>TestBed.configureTestingModule()</code>	Creates the isolated Angular environment
Control	<code>ComponentFixture<T></code>	Triggers CD, accesses DOM, reads component state
Async control	<code>fakeAsync + tick() / flushMicrotasks()</code>	Controls time and microtask queue without waiting

The key mental model shift: Angular's test change detection does not run automatically the way production change detection does. In tests, you must call `fixture.detectChanges()` manually to trigger a CD cycle. This gives you precise control over exactly when the DOM updates, making assertions reliable.

Section 2 — Connection to Prior Concepts

- Topic 25 — Component Lifecycle Hooks: TestBed triggers ngOnInit on the first fixture.detectChanges() call
- Topic 27 — Services & Dependency Injection: providing mock services via providers: [...] is DI in action
- Topic 43 — Angular Animations: provideNoopAnimations() in test providers prevents animation timing from breaking assertions
- Topic 44 — State Management Patterns: the primary thing you mock in component tests is the store service
- Topic 46 — RxJS Operators: fakeAsync + tick() is the correct way to test debounceTime and switchMap pipelines

Most directly extends: Topic 27 — Services & Dependency Injection (TestBed is DI in a controlled test environment).

Section 3 — Code Example

Full working component test

```
// counter.component.ts
@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <h1>{{ title() }}</h1>
    <p data-testid="count">{{ count() }}</p>
    <button data-testid="increment" (click)="increment()">+</button>
    <button data-testid="reset" (click)="reset()">Reset</button>
    @if (count() >= 10) {
      <span data-testid="warning">High count!</span>
    }
  `
})
export class CounterComponent {
  title = input<string>('Counter');
  count = signal(0);
  increment() { this.count.update(n => n + 1); }
  reset()      { this.count.set(0); }
}
```

```
// counter.component.spec.ts
describe('CounterComponent', () => {
  let fixture: ComponentFixture<CounterComponent>;
  let component: CounterComponent;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CounterComponent] // standalone goes in imports, not declarations
    }).compileComponents();

    fixture = TestBed.createComponent(CounterComponent);
    component = fixture.componentInstance;
    fixture.detectChanges(); // triggers ngOnInit + first render
  });

  it('should display the initial count as 0', () => {
    const countEl = fixture.debugElement.query(By.css('[data-testid="count"]'));
    expect(countEl.nativeElement.textContent.trim()).toBe('0');
  });

  it('should display the title input', () => {
    fixture.componentRef.setInput('title', 'My Custom Counter');
    fixture.detectChanges();
    const h1 = fixture.debugElement.query(By.css('h1'));
    expect(h1.nativeElement.textContent).toBe('My Custom Counter');
  });

  it('should increment count when + button is clicked', () => {
    const btn = fixture.debugElement.query(By.css('[data-testid="increment"]'));
    btn.nativeElement.click();
    fixture.detectChanges();
    const countEl = fixture.debugElement.query(By.css('[data-testid="count"]'));
    expect(countEl.nativeElement.textContent.trim()).toBe('1');
  });

  it('should show warning at count 10', () => {
    component.count.set(10);
    fixture.detectChanges();
    const warning = fixture.debugElement.query(By.css('[data-testid="warning"]'));
    expect(warning).not.toBeNull();
  });
});
```

Testing with a mock service

```

const mockUserService = {
  getCurrentUser: jasmine.createSpy('getCurrentUser').and.returnValue(
    of({ id: 1, name: 'Alice', email: 'alice@example.com' })
  )
};

beforeEach(async () => {
  await TestBed.configureTestingModule({
    imports: [UserProfileComponent],
    providers: [
      { provide: UserService, useValue: mockUserService }
    ]
  }).compileComponents();

  fixture = TestBed.createComponent(UserProfileComponent);
  fixture.detectChanges();
});

it('should display the user name', () => {
  const nameEl = fixture.debugElement.query(By.css('[data-testid="name"]'));
  expect(nameEl.nativeElement.textContent).toBe('Alice');
});

```

fakeAsync and tick() -- controlling time

```

it('should not call search before debounce period', fakeAsync(() => {
  const input = fixture.debugElement.query(By.css('[data-testid="search-input"]'));
  input.nativeElement.value = 'ang';
  input.nativeElement.dispatchEvent(new Event('input'));
  fixture.detectChanges();

  expect(mockSearchService.search).not.toHaveBeenCalled();

  tick(200); // still waiting -- needs 300ms total
  expect(mockSearchService.search).not.toHaveBeenCalled();

  tick(100); // 300ms total -- debounce fires
  expect(mockSearchService.search).toHaveBeenCalled();
}));

it('should only call search once for rapid typing', fakeAsync(() => {
  ['a', 'an', 'ang', 'angu', 'angul'].forEach(val => {
    input.nativeElement.value = val;
    input.nativeElement.dispatchEvent(new Event('input'));
    fixture.detectChanges();
    tick(50); // 50ms between keystrokes
  });
  tick(300); // debounce fires for the last value
  expect(mockSearchService.search).toHaveBeenCalledTimes(1);
  expect(mockSearchService.search).toHaveBeenCalledWith('angul');
}));

```

Section 4 — Before & After Refactor

Before — Testing implementation details

```

it('should set this.isLoading to true during fetch', () => {
  component.loadData();
  expect(component['isLoading']).toBe(true); // private implementation detail
});
it('should have 3 items in the array', () => {
  expect(component.items.length).toBe(3);
  // A broken template could render 0 items and this test still passes
});

```

What breaks: Tests pass even when the UI is broken because they're asserting on component internals. Refactoring internal variable names breaks these tests for no user-visible reason.

After — Testing user-visible behaviour

```

it('should show a loading spinner while data is fetching', fakeAsync(() => {
  fixture.detectChanges();
  const spinner = fixture.debugElement.query(By.css('[data-testid="spinner"]'));
  expect(spinner).not.toBeNull();
  tick(500);
  fixture.detectChanges();
  const spinnerAfter = fixture.debugElement.query(By.css('[data-testid="spinner"]'));
  expect(spinnerAfter).toBeNull();
}));

it('should render 3 item cards in the list', fakeAsync(() => {
  tick(500);
  fixture.detectChanges();
  const cards = fixture.debugElement.queryAll(By.css('[data-testid="item-card"]'));
  expect(cards.length).toBe(3);
}));

```

Section 5 — Common Mistakes

Mistake 1 — Forgetting fixture.detectChanges() after state changes

The mistake	component.count.set(5); then immediately asserting on DOM without detectChanges()
Why developers make it	In production, Angular's CD runs automatically. Developers assume tests work the same way
How to avoid it	After any state change -- signal.set(), property assignment, button click, timer tick -- call fixture.detectChanges()
Angular consequence	The test asserts on stale DOM output and fails intermittently

Mistake 2 — Using real timers instead of fakeAsync

The mistake	await new Promise(resolve => setTimeout(resolve, 300)) inside a test
-------------	--

Why developers make it	async/await is familiar; fakeAsync feels like unfamiliar Angular magic
How to avoid it	Use fakeAsync() => { tick(300); }) for timer-based async
Angular consequence	50 tests each waiting 300ms = 15 seconds of test suite time

Mistake 3 — Importing CommonModule or HttpClientModule into TestBed

The mistake	imports: [CommonModule, HttpClientModule, MyStandaloneComponent]
Why developers make it	In NgModule-based Angular, CommonModule was imported everywhere
How to avoid it	Standalone components declare their own imports -- TestBed inherits those automatically. Mock the service instead of importing HttpClientModule
Angular consequence	CommonModule bundles every Angular directive, slowing compilation. HttpClientModule is deprecated and cannot compose with provideHttpClient(withInterceptors([...]))

If the test genuinely needs real HttpClient (service tests only):

```
providers: [
  provideHttpClient(),           // modern functional provider
  provideHttpClientTesting()    // replaces HttpClientTestingModule
]
```

Section 6 — Do's and Don'ts

DO	DON'T
Add data-testid attributes to elements you need to query	Select by CSS class or element tag -- classes change with styling refactors
Call fixture.detectChanges() after every state change before asserting	Assert on DOM immediately after state changes without triggering CD
Use fakeAsync + tick() for all timer/debounce/delay testing	Use real setTimeout waits in tests
Mock services with { provide: RealService, useValue: mockObject }	Import HttpClientModule or HttpClientTestingModule in component tests

Test user-visible behaviour (DOM output, emitted events)	Access private component members via <code>component['privateProp']</code>
Use <code>fixture.componentRef.setInput()</code> for signal inputs	Set signal inputs via direct signal access -- they are read-only from outside
Use <code>provideHttpClient()</code> + <code>provideHttpClientTesting()</code> as providers in service tests	Use the legacy <code>HttpClientTestingModule</code> in <code>imports[]</code>
Test functional guards with <code>TestBed.runInInjectionContext(() => guardFn(route, state))</code>	Instantiate deprecated class guards with <code>new</code>

Section 7 — Interview Prep

Question 1 — Mid: "What is `ComponentFixture` and why do you need to call `detectChanges()` manually?"

Strong answer: `ComponentFixture<T>` is the test handle -- provides access to the instance, the DOM via `debugElement`, and ability to trigger CD. Tests don't use Zone.js automatic CD -- it would make assertions non-deterministic. Manual `detectChanges()` gives step-by-step control: set state -> trigger render -> assert. First `detectChanges()` triggers `ngOnInit` and the initial render.

Weak answers miss: The reason why CD is manual (determinism, testability of intermediate states).

Question 2 — Mid: "What is `fakeAsync` and when would you use it over a regular async test?"

Strong answer: `fakeAsync` creates a synchronous-appearing test zone where timers are controlled by `tick(ms)` instead of real time. Use for: any component with `debounceTime`, `timer()`, `setTimeout`, `setInterval`. `flush()` drains all pending timers at once. Use real `async/await` only for genuine Promise-based operations. 50 tests with `debounceTime(300)` take 0ms instead of 15 seconds.

Weak answers miss: The performance argument.

Question 3 — Senior: "How do you test a functional route guard in modern Angular?"

Strong answer: Functional guards are plain `CanActivateFn` functions -- no class to instantiate.

Correct pattern: `TestBed.runInInjectionContext(() => authGuard(mockRoute, mockState))`.

Provide mock dependencies via `configureTestingModule({ providers: [...] })`. Assert on the return value directly. `provideRouter([])` (not `RouterTestingModule`) is the minimal setup needed.

```
describe('authGuard', () => {
  const mockAuthStore = { isLoggedIn: signal(false) };

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        { provide: AuthStore, useValue: mockAuthStore },
        provideRouter([])
      ]
    });
  });

  it('should allow access when authenticated', () => {
    mockAuthStore.isLoggedIn.set(true);
    const result = TestBed.runInInjectionContext(() =>
      authGuard({} as ActivatedRouteSnapshot, {} as RouterStateSnapshot)
    );
    expect(result).toBe(true);
  });
});
```

Weak answers miss: That `TestBed.runInInjectionContext()` is the specific API for this pattern.

Section 8 — Quick Reference Card

3-line definition:

TestBed creates a real Angular environment in miniature -- real DI, real template compilation, real change detection -- with controlled, mockable dependencies. `ComponentFixture<T>` is the control interface; `fakeAsync + tick()` controls time. Nothing happens until you call `detectChanges()`, and everything external is a mock.

Task	How to do it
Import standalone component	<code>imports: [MyComponent]</code> in <code>configureTestingModule</code>
Provide a mock service	<code>providers: [{ provide: RealService, useValue: mock }]</code>
Trigger change detection	<code>fixture.detectChanges()</code>
Set a signal input	<code>fixture.componentRef.setInput('inputName', value)</code>
Query a single DOM element	<code>fixture.debugElement.query(By.css('[data-testid="x"]'))</code>
Trigger a click	<code>el.nativeElement.click()</code> then <code>fixture.detectChanges()</code>
Test with timers/debounce	<code>fakeAsync(() => { ...; tick(300); fixture.detectChanges(); })</code>
Drain all pending timers	<code>flush()</code> inside <code>fakeAsync</code>

Disable animations	provideNoopAnimations() in providers
Test a functional guard	TestBed.runInInjectionContext() => guardFn(route, state))
HTTP in service tests only	provideHttpClient() + provideHttpClientTesting() as providers

Scenario	Tool
debounceTime, setTimeout, timer()	fakeAsync + tick(ms)
All timers at once	fakeAsync + flush()
Recurring setInterval	fakeAsync + discardPeriodicTasks()
Real Promise resolution needed	waitForAsync + fixture.whenStable()

The one rule that matters most:

fixture.detectChanges() is not optional -- Angular's test environment never runs change detection automatically; every state change requires a manual detectChanges() call before DOM assertions are valid.

CDK & Animations

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The API surface is large and the mental model (states, transitions, triggers) requires a deliberate shift from thinking in CSS to thinking in Angular's declarative animation DSL; the mechanics are learnable but the layering of concepts trips up most developers on first contact.

Angular Relevance: High — Angular Animations is the production-correct way to animate components in an Angular app; it integrates with change detection, works with OnPush components, survives SSR, and avoids the brittle CSS-class-toggling patterns that break under Angular's rendering model.

Section 1 — Explanation

The Traffic Light Analogy

Imagine a traffic light. It has a fixed set of states (red, amber, green). At any moment it is always in exactly one state. When conditions change, it transitions from one state to another — and during that transition it does something visible. The transition itself has a duration and a curve.

Angular Animations work exactly like this:

- A state is a named snapshot of CSS properties (red = { backgroundColor: 'red' })
- A transition is the rule for moving between two states ('red => green') and how long it takes
- A trigger is the named container that groups states and transitions and is attached to a host element in the template

The template binds the trigger like a property: `[@trafficLight]="currentState"`. Whenever `currentState` changes in the component, Angular looks up the matching transition, calculates the interpolated CSS values, and plays the animation.

Technical Explanation

Angular Animations are powered by the Web Animations API and are provided through `@angular/animations`. As of modern Angular, registration should be done asynchronously via `provideAnimationsAsync()` rather than the older eager `provideAnimations()`, so the animations engine is not pulled into the initial bundle.

Function	Purpose	Where it lives
<code>trigger(name, [...])</code>	Names and groups states + transitions	<code>animations: []</code> metadata
<code>state(name, style({}))</code>	Defines CSS properties for a named state	Inside <code>trigger()</code>
<code>transition('a => b', [...])</code>	Defines what happens when moving between states	Inside <code>trigger()</code>

<code>animate('duration easing', style({}))</code>	Defines timing and optional end-style	Inside <code>transition()</code>
<code>keyframes([...])</code>	Multi-step animation with percentage checkpoints	Inside <code>animate()</code>
<code>query(selector, [...])</code>	Targets child elements inside a parent animation	Inside <code>transition()</code>
<code>stagger(delay, [...])</code>	Adds sequential delay between animated children	Inside <code>query()</code>
<code>group([...])</code>	Runs multiple animations in parallel	Inside <code>transition()</code>
<code>sequence([...])</code>	Runs multiple animations one after another	Inside <code>transition()</code>

The two special wildcard states: `*` matches any state (including `void`); `void` is the state of an element not in the DOM. This gives the two most common patterns:

- `'void => *'` = element enters the DOM (aliased as `:enter`)
- `'* => void'` = element leaves the DOM (aliased as `:leave`)

Section 2 — Connection to Prior Concepts

- Topic 25 — Component Lifecycle Hooks: `:enter` and `:leave` correspond to the component being added to and removed from the DOM
- Topic 24 — OnPush Change Detection: animation triggers bound in the template are evaluated during the same CD pass
- Topic 29 — Built-in Control Flow: `:enter/:leave` only work on elements toggled by `@if`; `[hidden]` keeps the element in the DOM
- Topic 17 — TypeScript Decorators: `@Component` metadata carries an `animations: []` array
- Topic 27 — Services & Dependency Injection: `provideAnimationsAsync()` is a DI provider; `inject()` is the only correct retrieval pattern for `AnimationBuilder`

Most directly extends: Topic 25 — Component Lifecycle Hooks (`:enter/:leave` map to component creation and destruction).

Section 3 — Code Example

Setup -- `provideAnimationsAsync()`

```
// app.config.ts
// MODERN: provideAnimationsAsync() lazy-loads the animation engine
// provideAnimations() (eager) is now legacy
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideAnimationsAsync() // required once at root level -- async variant
  ]
};
```

Fade in/out with named states

```
@Component({
  selector: 'app-notification',
  standalone: true,
  animations: [
    trigger('fadeInOut', [
      state('visible', style({ opacity: 1, transform: 'translateY(0)' })),
      state('hidden', style({ opacity: 0, transform: 'translateY(-10px)' })),
      transition('visible <=> hidden', [
        animate('300ms ease-in-out')
      ])
    ])
  ],
  template: `
    <div [@fadeOut]="panelState()" class="notification">Saved!</div>
    <button (click)="toggle()">Toggle</button>
  `
})
export class NotificationComponent {
  panelState = signal<'visible' | 'hidden'>('hidden');
  toggle() {
    this.panelState.set(this.panelState() === 'visible' ? 'hidden' : 'visible');
  }
}
```

Enter/leave animation with @if

```

@Component({
  standalone: true,
  animations: [
    trigger('slideFade', [
      transition(':enter', [
        style({ opacity: 0, transform: 'translateX(-20px)' }),
        animate('250ms ease-out', style({ opacity: 1, transform: 'translateX(0)' }))
      ]),
      transition(':leave', [
        animate('200ms ease-in', style({ opacity: 0, transform: 'translateX(20px)' }))
      ])
    ])
  ],
  template: `
    @if (showAlert()) {
      <!-- Angular holds the element in DOM until :leave animation completes -->
      <div @slideFade class="alert">Connection lost. Retrying...</div>
    }
    <button (click)="showAlert.set(!showAlert())">Toggle</button>
  `
})
export class AlertBannerComponent {
  showAlert = signal(true);
}

```

Staggered list animation

```

@Component({
  standalone: true,
  animations: [
    trigger('listAnimation', [
      transition('* => *', [
        query(':enter', [
          style({ opacity: 0, transform: 'translateY(10px)' }),
          stagger(60, [
            animate('300ms ease-out', style({ opacity: 1, transform: 'translateY(0)' }))
          ])
        ]), { optional: true }) // optional: true -- no error if no items entering
      ])
    ])
  ],
  template: `
    <!-- Trigger goes on the PARENT container, not individual items -->
    <ul [@listAnimation]="tasks().length">
      @for (task of tasks(); track task.id) { <li>{{ task.title }}</li> }
    </ul>
  `
})
export class TaskListComponent {
  private animationBuilder = inject(AnimationBuilder); // inject() -- not constructor
  tasks = signal<Task[]>([]);
}

```

Animated modal with group() + query()

```

@Component({
  selector: 'app-modal',
  standalone: true,
  animations: [
    trigger('modalAnimation', [
      transition(':enter', [
        group([
          query('.backdrop', [
            style({ opacity: 0 }),
            animate('200ms ease-out', style({ opacity: 1 })))
        ]),
          query('.panel', [
            style({ opacity: 0, transform: 'translateY(40px)' }),
            animate('250ms 50ms ease-out',
              style({ opacity: 1, transform: 'translateY(0)' })))
        ])
      ]),
      transition(':leave', [
        group([
          query('.backdrop', [animate('200ms ease-in', style({ opacity: 0 }))])),
          query('.panel', [animate('150ms ease-in', style({ opacity: 0, transform: 'translateY(20px)' })])
        ])
      ])
    ]),
  ],
  template: `
    @if (isOpen()) {
      <div @modalAnimation class="modal-host">
        <div class="backdrop" (click)="close()"></div>
        <div class="panel"><ng-content /></div>
      </div>
    }
  `
})
export class ModalComponent {
  isOpen = input.required<boolean>();
  closed = output<void>();
  close() { this.closed.emit(); }
}

```

Section 4 — Before & After Refactor

Before -- CSS class toggle animation


```

@Component({
  template: `
    <div [class.visible]="isVisible" class="panel">Content</div>
  `,
  styles: [`\`.panel { opacity: 0; transition: opacity 300ms; }
    .panel.visible { opacity: 1; }\`]
})
export class PanelComponent {
  isVisible = false;
  toggle() { this.isVisible = !this.isVisible; }
  // Combined with @if: @if removes from DOM before CSS transition plays
}

```

What breaks: CSS transition on `:leave` never plays because `@if` removes the element from DOM instantly. OnPush components may not detect the class change. Completely untestable via Angular's testing utilities.

After -- Angular animation trigger

```

@Component({
  standalone: true,
  animations: [
    trigger('fade', [
      transition(':enter', [
        style({ opacity: 0 }),
        animate('300ms ease-out', style({ opacity: 1 }))
      ]),
      transition(':leave', [
        animate('200ms ease-in', style({ opacity: 0 }))
      ])
    ])
  ],
  template: `
    @if (isVisible()) {
      <!-- Angular holds element in DOM until :leave animation finishes -->
      <div @fade class="panel">Content</div>
    }
    <button (click)="isVisible.set(!isVisible())">Toggle</button>
  `
})
export class PanelComponent {
  isVisible = signal(false);
}

```

Section 5 — Common Mistakes

Mistake 1 -- Forgetting `provideAnimationsAsync()` in `app.config.ts`

The mistake	Animations defined correctly but nothing animates; no error thrown
Why developers make it	Old tutorials use <code>BrowserAnimationsModule</code> in <code>AppModule</code> or the older <code>provideAnimations()</code>

How to avoid it	Check app.config.ts for provideAnimationsAsync(). Use provideNoopAnimations() in tests
Angular consequence	Animations silently do nothing. BrowserAnimationsModule pulls NgModule machinery into a standalone app. Eager provideAnimations() inflates the initial bundle

Mistake 2 -- Using [hidden] instead of @if for enter/leave animations

The mistake	<code><div @fade [hidden]="!isVisible">Content</div></code>
Why developers make it	[hidden] feels like toggling visibility; developers assume the animation engine watches visibility, not DOM presence
How to avoid it	:enter and :leave are triggered exclusively by elements being added/removed from the DOM. Only @if removes the element
Angular consequence	Element appears and disappears instantly -- :leave never fires because the element never leaves the DOM

Mistake 3 -- Putting the query() trigger on the child instead of the parent

The mistake	<code>@for (task of tasks(); ...) { <li [@listAnimation]="..."> }</code>
Why developers make it	It seems logical to animate each item individually
How to avoid it	The trigger must go on the container element; query(':enter', ...) finds all entering children within it
Angular consequence	Stagger never applies -- each item animates independently and simultaneously

Section 6 -- Do's and Don'ts

DO	DON'T
Call provideAnimationsAsync() once in app.config.ts	Use legacy BrowserAnimationsModule or eager provideAnimations()
Use @if to toggle element presence for :enter/:leave animations	Use [hidden] or display: none for enter/leave

Add { optional: true } to query() calls on lists that may be empty	Omit optional: true -- Angular throws if no matching children are found
Place the animation trigger on the parent container when using query() + stagger()	Place query()-based triggers on individual child elements
Use provideNoopAnimations() in test environments	Run full animation tests in every unit test
Use inject(AnimationBuilder) as a field initializer	Use constructor parameter injection for AnimationBuilder
Extract reusable triggers into a shared animations.ts constants file	Define large animation arrays inline in @Component metadata

Section 7 — Interview Prep

Question 1 — Mid: "What is the difference between :enter and :leave animations, and what must be true for them to fire?"

Strong answer: :enter = void => * -- fires when an element is added to the DOM. :leave = * => void -- fires when removed from the DOM. Element must be added/removed via @if -- [hidden] keeps the element in DOM and neither transition fires. Angular holds the element in DOM until :leave animation completes before destroying the view. provideAnimationsAsync() must be registered.

Weak answers miss: The critical @if requirement.

Question 2 — Mid/Senior: "How would you animate a list so that new items stagger in with a 60ms delay between each?"

Strong answer: query() + stagger() pattern -- trigger on the parent container, not individual items. The trigger value on the parent must change (bind to tasks().length). query(':enter', [...]) targets only newly added children. stagger(60, [animate(...)]) applies cumulative 60ms delay per child. { optional: true } prevents runtime error when list is empty.

Weak answers miss: That the trigger goes on the parent and { optional: true } is required.

Question 3 — Senior: "A leave animation was working in development but disappeared in the production build. What would you investigate?"

Strong answer: Check whether provideAnimationsAsync() is in the production app.config.ts vs. provideNoopAnimations() accidentally left in. Check whether @if is being used -- [hidden] silently skips :leave. Check for OnPush issues where the signal driving @if isn't updating correctly. Check for SSR: animations require provideNoopAnimations() on server and provideAnimationsAsync() only on client.

Weak answers miss: The SSR angle and the provideNoopAnimations() accidental override.

Section 8 — Quick Reference Card

3-line definition:

Angular Animations are a declarative state machine -- define named CSS states, declare transitions with timing, bind a trigger to a signal in the template, and Angular plays the correct animation automatically during change detection. `:enter/:leave` fire on DOM addition/removal via `@if`. `provideAnimationsAsync()` registers the engine once at root.

Need	Use
Animate between two named states	<code>state() + transition()</code>
Element enters or leaves the DOM	<code>:enter / :leave</code> transitions
Animate child elements with delay	<code>query() + stagger()</code>
Two animations at the same time	<code>group([...])</code>
Two animations one after the other	<code>sequence([...])</code>
Multi-step animation with checkpoints	<code>keyframes([...])</code>

The one rule that matters most:

`:leave` animations require `@if` -- `[hidden]` keeps the element in the DOM and the leave animation will never fire.

Performance Optimisation (trackBy, pure pipes, memoisation, deferrable views)

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Each individual technique is approachable, but the skill is knowing which problem each technique solves and when applying one without the others leaves performance on the table; the interactions between OnPush, signals, pure pipes, and deferrable views require a unified mental model — and that mental model now spans both Zone.js-based and zoneless change detection.

Angular Relevance: Critical — Performance problems in Angular almost always trace back to a small set of well-known patterns; knowing these techniques separates developers who build fast apps from those who build apps that feel sluggish at scale. This matters more, not less, as Angular moves toward zoneless change detection via `provideExperimentalZonelessChangeDetection()` — several of these techniques (notably OnPush and signal-driven rendering) are not just performance optimisations under zoneless, they become load-bearing correctness requirements, because there is no Zone.js fallback silently re-checking everything for you.

Section 1 — Explanation

The Restaurant Kitchen Analogy

Imagine a restaurant kitchen during a busy service. An inefficient kitchen re-cooks every dish from scratch every time the head chef walks through — even if the dish hasn't been ordered again. A well-run kitchen:

- Only re-cooks dishes that were actually ordered (OnPush change detection)
- Remembers the recipe result so identical orders skip the cooking step (pure pipes + memoisation)
- Doesn't start cooking the dessert menu until a table actually orders dessert (deferrable views)
- Knows which specific dish changed on a tray instead of replacing the whole tray (track)

Angular's legacy default behaviour -- Zone.js-driven, `ChangeDetectionStrategy.Default` -- is the inefficient kitchen: the head chef (Zone.js) walks the entire restaurant after every single event and re-checks every table. Each technique in this topic is one discipline that makes the kitchen run efficiently regardless of which change detection model is running underneath.

The two change detection models

	Zone.js-based (current default)	Zoneless (<code>provideExperimentalZonelessChangeDetection()</code>)
--	---------------------------------	--

What triggers a CD check	Zone.js patches async APIs and schedules a check after each one	Signal writes and explicit notifications schedule a check directly
Cost of unoptimised Default component	High -- re-checked on every async event anywhere in the app	Even higher -- without Zone.js safety net, components may not update reliably
Role of OnPush	Performance optimisation -- skips redundant checks	Closer to a requirement -- no Zone.js broadly re-checking as fallback
Role of signals	Recommended for clean state	Foundational -- signal writes are what schedule the check

The four techniques

Technique	Problem it solves	Where it applies
track	DOM thrashing when lists re-render	@for loops with dynamic data
Pure pipes	Redundant transform recalculation	Template expressions that transform data
Memoisation	Expensive function re-execution	Component methods called in templates
@defer	Blocking render with off-screen content	Below-the-fold or conditionally shown components

Section 2 — Connection to Prior Concepts

- Topic 24 — OnPush Change Detection: every technique here is an extension of the OnPush mental model; under zoneless this becomes the default assumption, not an opt-in optimisation
- Topic 42 — Custom Pipes: pure pipes as a performance tool; this topic quantifies the impact
- Topic 29 — Built-in Control Flow: @for with track and @defer are both built-in control flow features
- Topic 23 — Angular Signals: signals replace the need for trackBy in some scenarios and are the underlying primitive for zoneless change detection
- Topic 44 — State Management Patterns: stores provide the immutable data that makes track and pure pipes work correctly

Bridging note: In Topic 24 you learned that OnPush skips a component's re-render unless its inputs change by reference. The techniques in this topic extend that discipline further down the rendering stack. Under Zone.js, skipping these techniques costs you performance. Under zoneless change detection, skipping OnPush and signal-driven inputs specifically can cost you correctness -- a component that mutates state outside the signal graph may simply never re-render.

Most directly extends: Topic 24 -- OnPush Change Detection (all four techniques are downstream optimisations of the same reference-equality mental model).

Section 3 — Code Example

Technique 1 -- track in @for

```
// Without track -- Angular destroys and recreates every DOM node on every list update
@for (item of items(); ) {
  <app-item [data]="item" />
}

// With track -- Angular matches existing DOM nodes to new data by identity
@for (item of items(); track item.id) {
  <app-item [data]="item" />
}

// When track $index is acceptable vs. when it breaks:
// ACCEPTABLE: static list that never reorders, no child component state
@for (label of ['Home', 'About', 'Contact']; track $index) { <li>{{ label }}</li> }

// BREAKS: dynamic list where items reorder or have child components with state
// index 0 might point to a different item after sort -- child component
// retains its previous internal state while receiving new input() data
@for (user of sortedUsers(); track $index) { <app-user-card [user]="user" /> }

// ALWAYS CORRECT: stable unique ID
@for (user of sortedUsers(); track user.id) { <app-user-card [user]="user" /> }
```

Technique 2 -- Pure pipes as a performance tool

```
// PROBLEM: method calls in templates run on EVERY change detection cycle
getFilteredItems(): Item[] { // called on every mousemove, keydown, timer tick
  return this.items().filter(i => i.name.includes(this.filter()));
}

// FIX: pure pipe -- Angular memoises, only recalculates when input references change
@Pipe({ name: 'filterItems', standalone: true, pure: true })
export class FilterItemsPipe implements PipeTransform {
  transform(items: Item[] | null, query: string): Item[] {
    if (!items) return [];
    if (!query) return items;
    return items.filter(i => i.name.toLowerCase().includes(query.toLowerCase()));
  }
}

// Template usage:
// @for (item of (items() | filterItems : filter()); track item.id) { ... }
```

Technique 3 -- Memoisation with computed()

```

@Injectables({ providedIn: 'root' })
export class ReportStore {
  readonly #transactions = signal<Transaction[]>([]);

  // computed() memoises -- only recalculates when #transactions signal changes
  readonly monthlyTotals = computed(() => {
    this.#transactions().reduce((acc, t) => {
      const month = t.date.slice(0, 7);
      acc[month] = (acc[month] ?? 0) + t.amount;
      return acc;
    }, {} as Record<string, number>)
  });

  readonly topCategory = computed(() => {
    const counts = this.#transactions().reduce((acc, t) => {
      acc[t.category] = (acc[t.category] ?? 0) + 1;
      return acc;
    }, {} as Record<string, number>);
    return Object.entries(counts).sort((a, b) => b[1] - a[1])[0]?.[0] ?? 'none';
  });
}

```

Technique 4 -- Deferrable views (@defer)


```

@Component({
  standalone: true,
  template: `
    <!-- Above the fold: renders immediately, in the main bundle -->
    <app-hero-banner />
    <app-featured-products />

    <!-- Below the fold: deferred until user scrolls here -->
    @defer (on viewport) {
      <app-heavy-analytics-dashboard />
      @placeholder {
        <!-- Real dimensions prevent layout shift (CLS) -->
        <div class="skeleton" style="height: 400px;"></div>
      }
      @loading (minimum 300ms) { <app-spinner /> }
      @error { <p>Failed to load. <button (click)="retry()">Retry</button></p> }
    }

    <!-- Multiple triggers + prefetch -->
    @defer (on idle; on viewport; prefetch on hover) {
      <app-recommendations />
      @placeholder { <div class="skeleton"></div> }
    }

    <!-- Signal-driven manual control -->
    @defer (when showComments()) {
      <app-comment-thread [postId]="postId()" />
      @placeholder { <div>Click to load comments</div> }
    }
  `
})
export class HomePageComponent {
  postId      = input.required<number>();
  showComments = signal(false);
}

```

Section 4 — Before & After Refactor

Before -- Unoptimised list component

```

@Component({
  changeDetection: ChangeDetectionStrategy.Default,
  template: `
    @for (product of getFilteredProducts(); ) {
      <app-product-card [product]="product" />
    }
    <app-sales-chart [data]="salesData" />
  `
})
export class ProductListComponent {
  products: Product[] = [];
  searchQuery = '';

  getFilteredProducts(): Product[] {
    // Runs on every mousemove -- 1,000 filter evaluations per event
    return this.products.filter(p =>
      p.name.toLowerCase().includes(this.searchQuery.toLowerCase())
    );
  }
}

```

After -- All four optimisations applied

```

@Pipe({ name: 'filterProducts', standalone: true, pure: true })
export class FilterProductsPipe implements PipeTransform {
  transform(products: Product[], query: string): Product[] {
    if (!products) return [];
    if (!query) return products;
    const lower = query.toLowerCase();
    return products.filter(p => p.name.toLowerCase().includes(lower));
  }
}

@Component({
  standalone: true,
  imports: [FilterProductsPipe],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (product of (products() | filterProducts : searchQuery()); track product.id) {
      <app-product-card [product]="product" />
    }
    @defer (on viewport) {
      <app-sales-chart [data]="salesData()" />
      @placeholder { <div class="chart-skeleton" style="height:300px"></div> }
      @loading { <app-spinner /> }
    }
  `
})
export class ProductListComponent {
  products = input.required<Product[]>();
  salesData = input.required<SalesData[]>();
  searchQuery = signal('');
}

```

Section 5 — Common Mistakes

Mistake 1 -- Using track \$index on dynamic lists

The mistake	@for (user of sortedUsers(); track \$index) on a sortable list
Why developers make it	\$index is available automatically and silences linting warnings
How to avoid it	track \$index is only safe when the list never reorders and items have no internal component state
Angular consequence	When the list reorders, the DOM node for index 0 stays alive and receives new input() data but retains the previous item's internal state -- visible data and component state silently diverge

Mistake 2 -- Treating @defer as a substitute for all performance work

The mistake	Wrapping everything in @defer, skipping OnPush, track, and pure pipes inside the deferred block
Why developers make it	@defer is visually dramatic -- clearly splits the bundle and feels like "doing performance work"
How to avoid it	@defer delays initial load only. All four techniques are complementary and address different phases
Angular consequence	The app loads fast but becomes sluggish as soon as the deferred component renders

Mistake 3 -- Calling expensive methods in templates instead of computed() or pure pipes

The mistake	{{ formatReport(reportData) }} -- method call in template
Why developers make it	Method calls are convenient; developers don't realise Angular cannot memoise them
How to avoid it	Any template expression that transforms data should be a pure pipe or a computed() signal
Angular consequence	With 50 rows each calling two methods, that's 100 function calls per CD cycle -- measurable frame drops

Section 6 -- Do's and Don'ts

DO	DON'T
----	-------

Use track item.id (stable entity ID) on all dynamic lists	Use track \$index on lists that reorder, sort, or filter
Use pure pipes for all data transformation in templates	Call methods that return transformed data directly from templates
Use computed() for all derived state in signal-based components	Re-derive calculated values inside component methods called from templates
Apply @defer (on viewport) to any significant component below the fold	Render every component eagerly regardless of initial visibility
Combine all four techniques -- they are complementary	Apply only one technique and consider performance "done"
Use @placeholder with skeleton UI matching deferred component dimensions	Use an empty @placeholder -- without it, layout shift (CLS) degrades Core Web Vitals
Treat OnPush + signals as mandatory under provideExperimentalZonelessChangeDetection()	Mutate plain class properties expecting Zone.js to catch the update -- under zoneless it won't

Section 7 — Interview Prep

Question 1 — Mid: "Why can using track \$index in an @for loop cause bugs, and when is it actually safe?"

Strong answer: track \$index uses the array position as the DOM node identity. When the list reorders, the index of each item changes but the DOM node stays -- Angular updates input() on the existing component but the component's internal state belongs to the previous item. Safe only for: static lists that never reorder, primitive value lists with no child component state. Always prefer track item.id.

Weak answers miss: The internal component state problem.

Question 2 — Mid/Senior: "What is the difference between @defer (on viewport) and route-based lazy loading?"

Strong answer: Route lazy loading splits at page/route boundaries -- triggered by navigation. @defer splits at component boundaries within a page -- triggered by viewport, idle, interaction, or a signal condition. They are complementary -- a lazy-loaded route can contain @defer blocks for further within-page splitting. @defer also handles the render lifecycle (placeholder, loading, error) which route lazy loading does not. Use route splitting for separate pages; use @defer for heavy content within a page.

Weak answers miss: That the two mechanisms compose.

Question 3 — Senior: "An Angular app is slow during user input in a search box. The component uses OnPush. What would you investigate?"

Strong answer: Check for template method calls that return filtered/sorted arrays -- these run every CD cycle even with OnPush. Replace method calls with a pure pipe -- skips recalculation if the query string hasn't changed. Check track on any @for loops in search results -- without it, every keystroke rebuilds the entire DOM. Check whether child components also lack OnPush.

Mention `debounceTime` on the search input. Mention `@defer` for heavy components in results not immediately visible.

Weak answers miss: The pure pipe fix -- most candidates jump to `debounceTime` without identifying the root cause.

Section 8 — Quick Reference Card

3-line definition:

Angular performance is about doing less work at every phase -- less to load (`@defer`), less to render (`OnPush` + signals), less to compute (pure pipes + `computed()`), and less DOM to patch (track). Each technique targets a different phase of the rendering pipeline. Under zoneless change detection, `OnPush` and signal-driven state stop being optional performance choices and become correctness requirements.

Technique	When it runs	What it prevents	Key rule
<code>track item.id</code>	During list reconciliation	Full DOM rebuild on every update	Never use <code>\$index</code> on sortable/filterable lists
Pure pipe	During CD, input changes only	Redundant transform recalculation	Returns memoised result when input reference unchanged
<code>computed()</code>	When signal dependencies change	Redundant expensive derivation	Only recalculates when tracked signals emit
<code>@defer</code>	On trigger (viewport, idle, interaction)	Eager render of off-screen content	Always give <code>@placeholder</code> real dimensions

Symptom	Root cause	Fix
Slow typing / input lag	Method calls in template every CD cycle	Pure pipe or <code>computed()</code>
List flicker on update	No track -- full DOM rebuild	<code>track item.id</code>
Slow initial page load	Heavy components in main bundle	<code>@defer</code> (on viewport)
Page layout shift on load	Empty <code>@placeholder</code>	Give <code>@placeholder</code> real dimensions
Child components not updating	<code>track \$index</code> on reordering list	<code>track item.id</code>
View silently never updates	State mutated outside signal graph (zoneless)	Route state through <code>signal()/update()</code> ; verify with Angular DevTools

The one rule that matters most:

Template method calls that transform data are never memoised -- replace them with pure pipes or computed() signals; this is the single most common source of Angular performance problems in production, and under zoneless change detection, signal-driven state is not optional -- it's how the view learns anything changed at all.